

Signals

1:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
void handle_signal(int sig)
{
    printf("\nSIGINT received!\n");
}
int main()
{
    signal(SIGINT, handle_signal);
    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}
```

Output:

```
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received!
Program is running...
Program is running...
^C
SIGINT received!
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received!
Program is running...
Program is running...
```

2:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
while (1)
{
printf("Program is running...\n");
sleep(2);
}
return 0;
}.
```

Output:

```
nishanth@LAPTOP-6Q3ON07H:~/signals$ ./signal2
Program is running...
Program is running...
Program is running...
Program is running...
^C
```

3:

```
GNU nano 2.7.1
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handle_sigchld(int sig)
{
printf("SIGCHLD received: Child has terminated.\n");
}
int main()
{
signal(SIGCHLD, handle_sigchld);
pid_t pid = fork();
if (pid == 0)
{
// Child
printf("Child is running...\n");
sleep(3);
printf("Child is terminating...\n");
}
else
{
// Parent
printf("Parent is waiting...\n");
wait(NULL);
printf("Parent finished.\n");
}
return 0;
}
```

Output:

```
nishanth@LAPTOP-6Q3ON07H:~/signals$ gcc signal3.c -o signal3
nishanth@LAPTOP-6Q3ON07H:~/signals$ ./signal3
Parent is waiting...
Child is running...
Child is terminating...
SIGCHLD received: Child has terminated.
Parent finished.
```

4:

```
GNU nano 2.7.1
#include <stdio.h>
#include <unistd.h>
int main()
{
    int fd[2];
    pipe(fd);
    close(fd[0]); // Close read end
    printf("Writing to pipe...\n");
    write(fd[1], "Hello", 5);
    printf("Write completed.\n");
    return 0;
}
```

Output:

```
nishanth@LAPTOP-6Q3ON07H:~/signals$ gcc signal4.c -o signal4
nishanth@LAPTOP-6Q3ON07H:~/signals$ ./signal4
Writing to pipe...
```

5:

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
void handler(int sig)
{
printf("Child received SIGUSR1\n");
}
int main()
{
pid_t pid = fork();
if (pid == 0)
{
// Child
signal(SIGUSR1, handler);
printf("Child waiting...\n");
while (1)
{
sleep(1);
}
}
else
{
// Parent
sleep(2);
printf("Parent sending signal...\n");
kill(pid, SIGUSR1);
sleep(2);
kill(pid, SIGTERM);
wait(NULL);
}
return 0;
}

```

Output:

```

nishanth@LAPTOP-6Q3ON07H:~/signals$ gcc signal5.c -o signal5
nishanth@LAPTOP-6Q3ON07H:~/signals$ ./signal5
Child waiting...
Parent sending signal...
Child received SIGUSR1

```

6:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
printf("Program started\n");
abort();
printf("Program ended\n");
return 0;
}.
```

Output:

```
nishanth@LAPTOP-6Q3ON07H:~/signals$ ./signal6
Program started
Aborted ./signal6
```

7:

```
GNU nano 2.7.1
#include <stdio.h>
int main()
{
int *p = NULL;
printf("Before error\n");
*p = 10;
printf("After error\n");
return 0;
}.
```

Output:

```
nishanth@LAPTOP-6Q3ON07H:~/signals$ ./sig
Before error
Segmentation fault ./signal7
nishanth@LAPTOP-6Q3ON07H:~/signals$ .
```